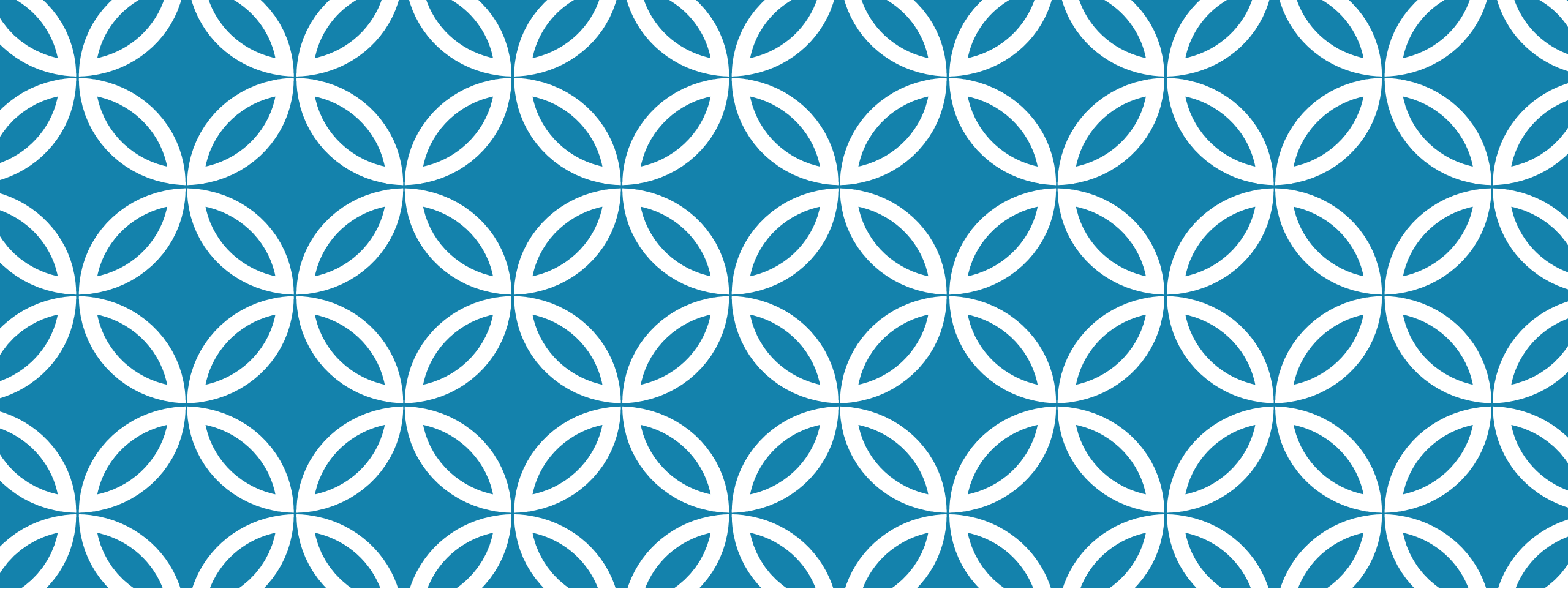




CRÉATION D'UNE APPLICATION CLOUD NATIVE



INTRODUIRE LE CLOUD NATIVE

C'est quoi le cloud?

Les avantages du cloud

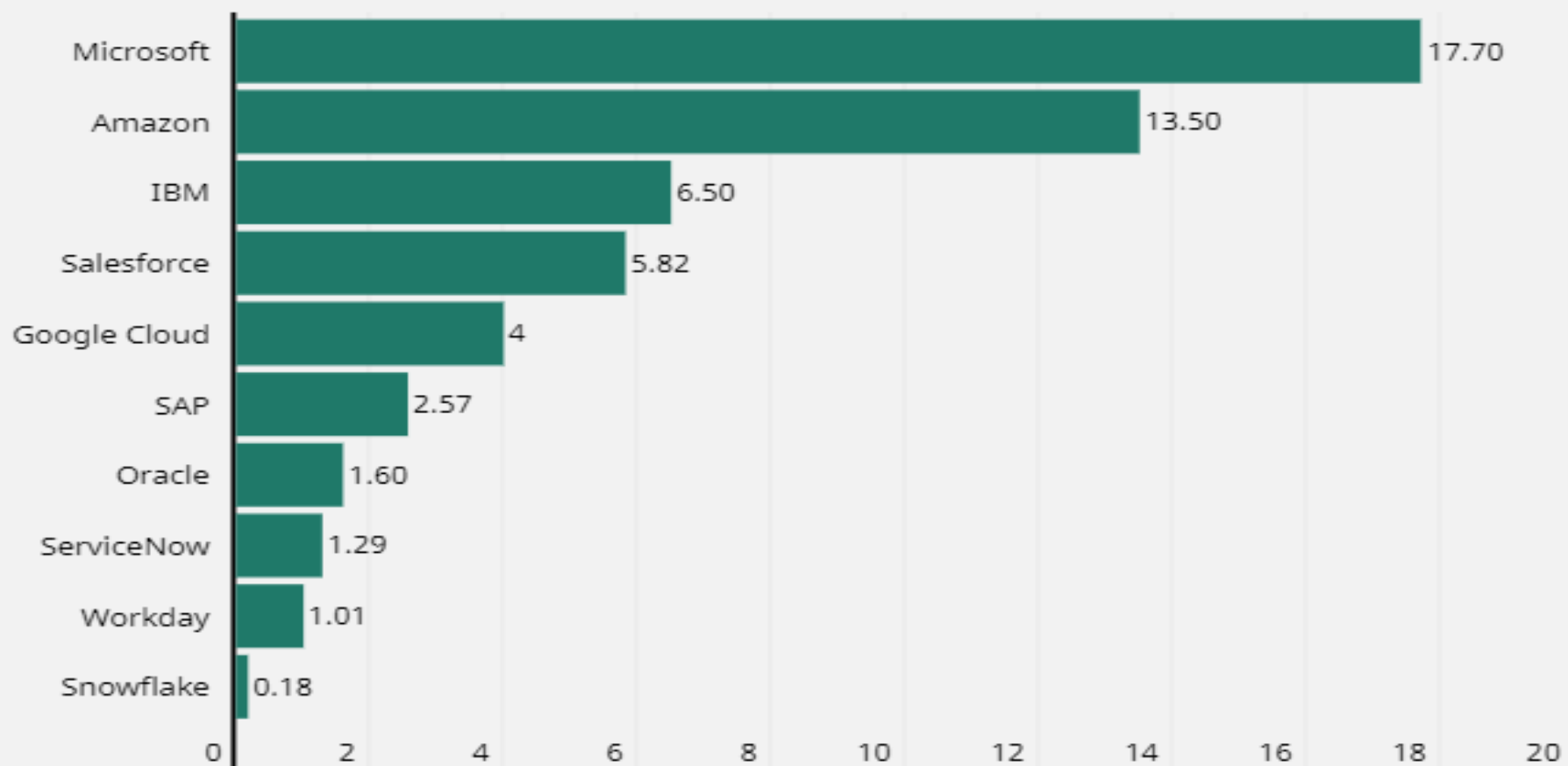
Les fournisseurs du cloud

Les différents types de Cloud

Les services du cloud (IaaS, PaaS, SaaS).

FOURNISSEURS CLOUD

Source: Statista, cloudwars.co



EXEMPLE UTILISATION CLOUD

accéder aux mêmes fichiers et aux mêmes applications à partir de presque n'importe quel appareil

vous pouvez vous connecter votre compte Instagram à partir de n'importe quel appareil, avec toutes vos photos, vidéos et l'historique de vos conversations. Il en va de même avec les fournisseurs de messagerie cloud comme Gmail ou Microsoft Office 365 et les fournisseurs de stockage cloud comme Dropbox ou Google Drive.

Pour les entreprises, le passage au cloud computing supprime certains coûts et frais informatiques : par exemple, les sociétés n'ont plus besoin de mettre à jour et d'entretenir leurs propres serveurs, c'est le fournisseur de cloud qui s'en charge

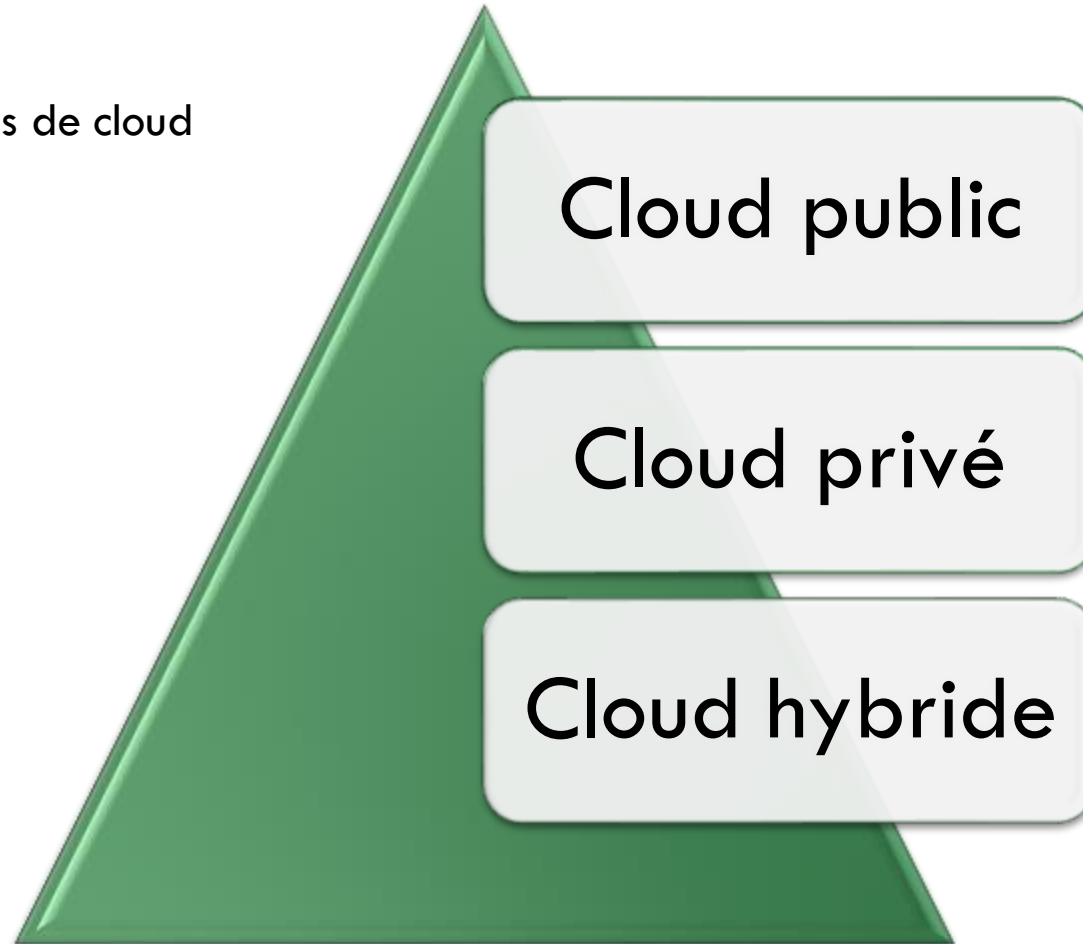
SERVEUR INFORMATIQUE VS CLOUD PRIVÉ

quelle solution de stockage de données choisir pour une entreprise ?

Avantages et inconvénients de chacun

DIFFÉRENTS TYPES DE CLOUD

Donner les différents types de cloud que vous connaissez.



CLOUD PUBLIC

Les clouds **publics** sont généralement des environnements cloud créés à partir d'une infrastructure informatique qui n'appartient pas à l'utilisateur final.

Alibaba Cloud, Microsoft Azure, Google Cloud, Amazon Web Services (AWS) et IBM Cloud sont les principaux fournisseurs de cloud public.



CLOUD PRIVE

Les clouds **privés** sont des environnements cloud spécifiques à un utilisateur final ou à un groupe, et sont habituellement exécutés derrière le pare-feu de l'utilisateur ou du groupe.

Tous les clouds deviennent des clouds **privés** lorsque l'infrastructure informatique sous-jacente est spécifique à **un client unique**, avec un accès entièrement **isolé**.



CLOUD PRIVÉ

Aujourd'hui, les entreprises créent des clouds privés dans des **datacenters hors site** et loués à des fournisseurs, ce qui rend les règles relatives à l'emplacement et à la propriété obsolètes.

Cette tendance a fait naître différents sous-types de clouds privés, notamment :

- ✓ Clouds privés gérés: Ce type de cloud est créé et utilisé par les clients, tandis qu'il est déployé, configuré et géré par un fournisseur tiers.
- ✓ Clouds dédiés: Il s'agit d'un cloud au sein d'un autre cloud. Vous pouvez déployer un cloud spécialisé dans un cloud public.

CLOUD HYBRIDE

Un cloud **hybride** fonctionne comme un environnement informatique unique créé à partir de plusieurs environnements connectés via des réseaux locaux (LAN), des réseaux étendus (WAN), des réseaux privés virtuels (VPN) et/ou des API.

Les caractéristiques des clouds hybrides sont complexes et les exigences associées peuvent varier selon l'utilisateur qui les définit. Par exemple, un cloud hybride peut inclure :

- ✓ Au moins un cloud privé et au moins un cloud public
- ✓ Au moins deux clouds privés
- ✓ Au moins deux clouds publics
- ✓ Un environnement virtuel connecté à au moins un cloud privé ou public

LES DIFFÉRENTS SERVICES DU CLOUD

Regarder la vidéo et extraire les différents types de services et leurs propriétés:



LES DIFFÉRENTS SERVICES DU CLOUD

Chaque type de cloud computing allège la gestion de votre infrastructure sur site.

Il existe trois principaux types de cloud computing « **as-a-Service** », chacun offrant un certain degré de gestion :

- **IaaS** (Infrastructure-as-a-Service)
- **PaaS** (Platform-as-a-Service)
- **SaaS** (Software-as-a-Service).

On-site	IaaS	PaaS	SaaS
Applications	Applications	Applications	Applications
Data	Data	Data	Data
Runtime	Runtime	Runtime	Runtime
Middleware	Middleware	Middleware	Middleware
O/S	O/S	O/S	O/S
Virtualization	Virtualization	Virtualization	Virtualization
Servers	Servers	Servers	Servers
Storage	Storage	Storage	Storage
Networking	Networking	Networking	Networking

 You manage

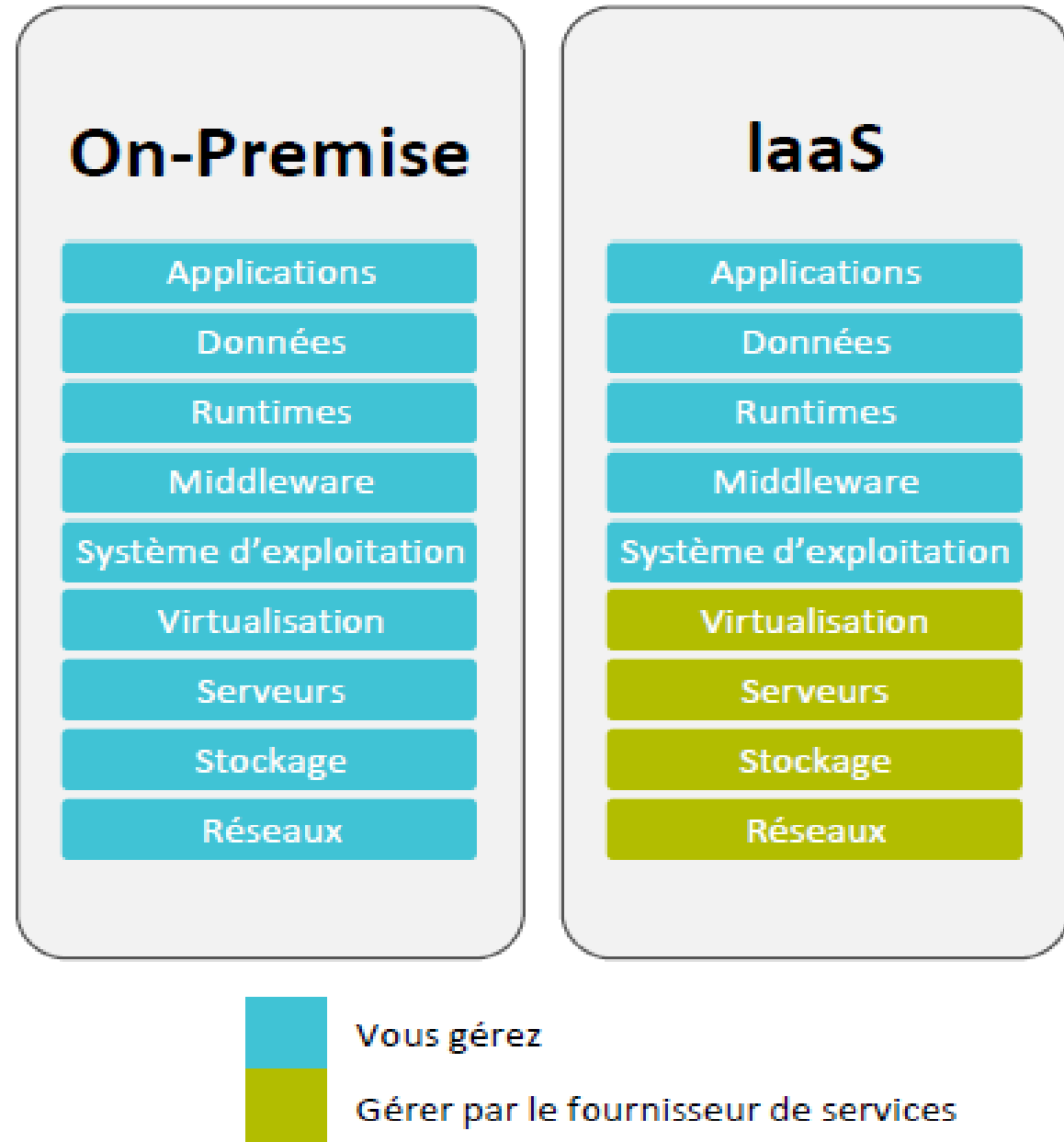
 Service provider manages

INFRASTRUCTURE AS A SERVICE

l'utilisateur ne contrôle pas l'infrastructure Cloud sous jacente et il n'a pas à s'inquiéter des mises à jour physiques ou de la maintenance de ces composants;

Par contre et en tant qu'utilisateur:

vous êtes responsable du **système d'exploitation** ainsi que **des données applications**, solutions de middleware et environnements d'exécution.

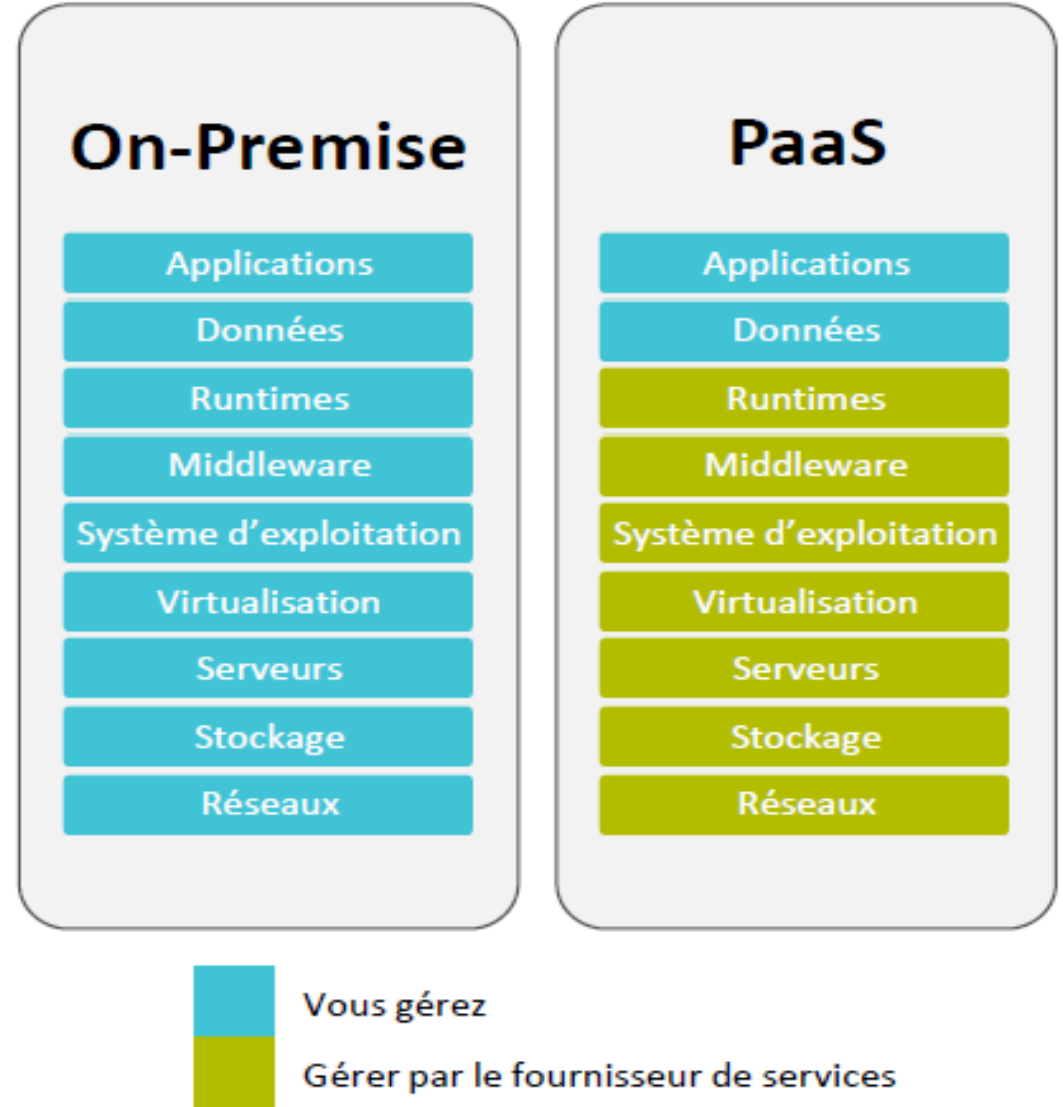


PAAS : PLATFORM-AS-A-SERVICE

l'utilisateur ne contrôle pas l'infrastructure Cloud sous-jacente et il n'a pas à s'inquiéter des mises à jour physiques ou de la maintenance de ces composants y compris le réseaux, les serveurs, les systèmes d'exploitations ou de stockage.

Par contre et en tant qu'utilisateur:

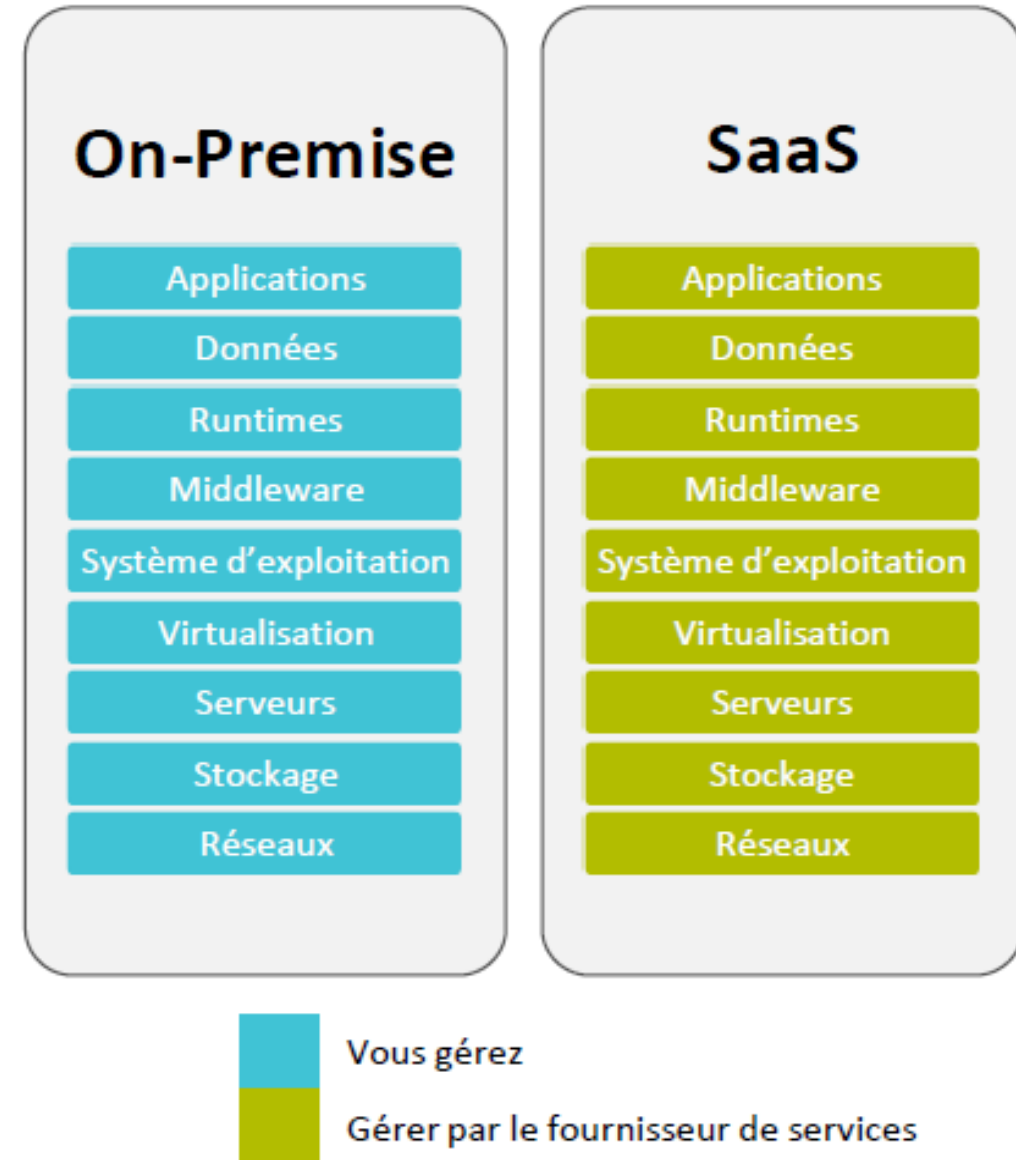
vous avez le contrôle pour le déploiement et configuration d'applications créées à l'aide de langages de programmation, de bibliothèques, de services et d'outils pris en charge par le fournisseur.



SAAS SOFTWARE AS A SERVICE

Pour ce type de service le fournisseur fournit et gère une application complète accessible par les utilisateurs via un navigateur Web ou un client lourd

Ainsi, l'utilisateur ne contrôle pas la plateforme Cloud sous jacente et il n'a pas à s'inquiéter des mises à jour logicielles ou l'application des correctifs et les autres tâches de maintenance logicielle



SAAS SOFTWARE AS A SERVICE

Pour ce type de service le fournisseur fournit et gère une application complète accessible par les utilisateurs via un navigateur Web ou un client lourd

Ainsi, l'utilisateur ne contrôle pas la plateforme Cloud sous jacente et il n'a pas à s'inquiéter des mises à jour logicielles ou l'application des correctifs et les autres tâches de maintenance logicielle

Fournisseurs	AWS	Google Cloud	Azure
SaaS services	Zoom 	Google Apps 	Microsoft Office 365 

CHAPITRE2: L'APPROCHE CLOUD NATIVE

- ❑ Définition de l'approche cloud native
- ❑ Avantages
- ❑ Vue générale sur les caractéristiques du cloud natif :
 - ✓ Automatisation des processus du développement et de déploiement,
 - ✓ Microservices et conteneurs.

DÉFINITION DE L'APPROCHE CLOUD NATIVE

Cloud Native : le Cloud Native décrit une approche de développement logiciel dans laquelle les applications sont dès le début conçues pour une utilisation sur le Cloud.

Il en résulte des applications Cloud Native (NCA) capables de pleinement exploiter les atouts de l'architecture du *Cloud Computing*.

Cette approche se concentre sur le **développement d'applications sous la forme de microservices individuels**, qui ne sont pas exécutés « On-Premises » (localement), mais sur **des plateformes agiles** basées sur **des conteneurs**.

Cette approche accélère le développement de logiciels et favorise la création d'applications **résilientes** et **évolutives**.

FONCTIONNEMENT

L'approche Cloud Native repose sur **quatre piliers** qui sont liés et interdépendants.

- Du côté **technique**, on trouve les **microservices** et les technologies de **conteneurs** développées spécialement pour l'environnement Cloud qui constituent des éléments fondamentaux du concept Cloud Native. Les différents microservices remplissent une fonction précise et peuvent être rassemblés dans un conteneur avec tout ce qui est nécessaire à leur exécution. Ces conteneurs sont portables et offrent aux équipes de développement un haut degré de flexibilité, par exemple lorsqu'il s'agit de tester de nouveaux services.
- Du côté de la **stratégie**, les **processus de développement** et la **Continuous Delivery** sont bien établis. Lors de la conception d'une architecture Cloud Native efficace, les équipes de développeurs (Developers = Dev), mais aussi l'entreprise (Operations = Ops) sont directement impliquées. Dans le cadre d'un échange constant, l'équipe de développeurs ajoute à un microservice certaines fonctionnalités livrées automatiquement par des processus de **Continuous-Delivery**.

- ✓ Voici quelques entreprises qui ont implémenté des techniques natives Cloud et qui ont obtenu, par conséquence, la vitesse, l'agilité et la scalabilité.
- ✓ **Netflix, Uber et WeChat** exposent des systèmes natifs Cloud qui se composent de nombreux services indépendants. Ce style architectural leur permet de répondre rapidement aux conditions du marché. Elles mettent instantanément à jour de petites zones d'une application complexe en service, sans redéploiement complet. Elles mettent à l'échelle individuellement les services en fonction des besoins

Entreprise	Expérience
	Dispose de plus de 600 services en production. Effectue des déploiements 100 fois par jour.
	Dispose de plus de 1 000 services en production. Effectue des déploiements plusieurs milliers de fois par semaine.
	Dispose de plus de 3 000 services en production. Effectue des déploiements 1 000 fois par jour.

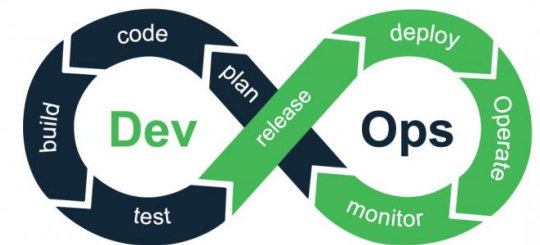
VUE GÉNÉRALE SUR LES CARACTÉRISTIQUES

AUTOMATISATION DES PROCESSUS DU DÉVELOPPEMENT ET DE DÉPLOIEMENT:

Comme l'approche DevOps, le **Cloud Native** cherche à rassembler les équipes Dev et Ops autour d'un objectif commun long terme : celui de la création de valeur business par les applications.

L'approche DevOps permet de converger vers une **approche Cloud Native** avec l'automatisation des processus et des technologies entre les équipes, de façon à intégrer plus rapidement les innovations dans les cycles de développement et de déploiement d'une **application Cloud Native**.

En parallèle du **Cloud Native**, l'adoption des méthodes Agiles va permettre d'intégrer les équipes métier dans cette collaboration avec les équipes techniques et de développement. L'idée est de collaborer pour délivrer une itération en améliorant le produit à chaque livraison de façon continue.



LES MICROSERVICES

Les microservices désignent à la fois une architecture et une approche de développement logiciel qui consiste à décomposer les applications en éléments les plus simples, indépendants les uns des autres. Contrairement à une approche monolithique classique, selon laquelle tous les composants forment une entité indissociable, les microservices fonctionnent en synergie pour accomplir les mêmes tâches, tout en étant séparés.

Pour communiquer entre eux, les **microservices** d'une application utilisent le modèle de communication requête-réponse. L'implémentation typique utilise des appels API REST basés sur le protocole HTTP. Les procédures internes (appels de fonctions) facilitent la communication entre les composants de l'application.

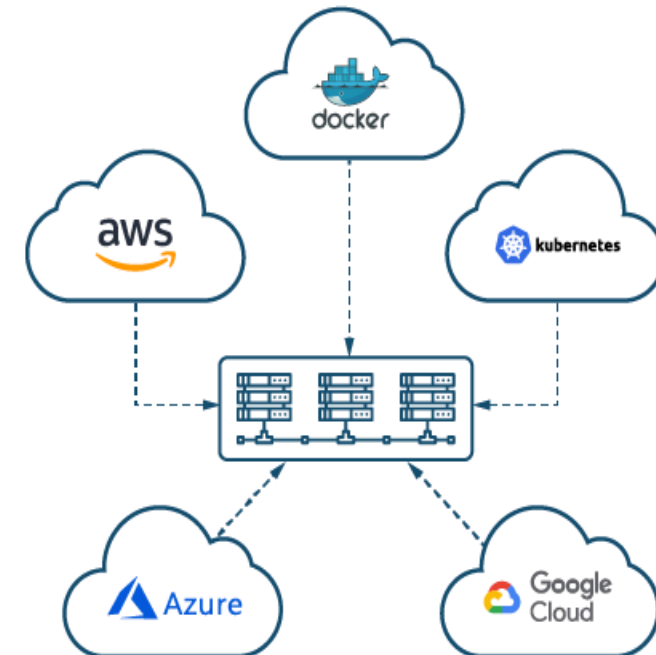
les microservices sont beaucoup plus faciles à créer,
tester, déployer et mettre à jour

LES CONTENEURS

Tout comme le secteur du transport utilise des conteneurs pour isoler les différentes marchandises à transporter à bord des navires, des trains, des camions et des avions, le développement logiciel a de plus en plus recours au concept de **conteneurisation**.

Un package logiciel unique, appelé « **conteneur** », regroupe le code d'une application avec les fichiers de configuration, les bibliothèques et les dépendances requises pour que l'application puisse s'exécuter.

Ceci permet aux développeurs et aux professionnels de l'informatique de déployer les applications de façon **transparente dans tous les environnements**.





CRÉER DES APIS REST SIMPLES EN NODE JS ET EXPRESS JS

C'EST QUOI UNE API REST

Une API (interface de programmation d'application) est un ensemble de définitions et de protocoles qui facilite la création et l'intégration de logiciels d'applications.

l'API est **l'intermédiaire** permettant à deux systèmes informatiques totalement indépendants d'interagir entre eux, de manière automatique, sans intervention humaine.

C'EST QUOI UNE API REST

Elle est parfois considérée comme un **contrat** entre un **fournisseur** d'informations et un **utilisateur** d'informations, qui permet de définir le contenu demandé au consommateur (l'appel) et le contenu demandé au producteur (la réponse).

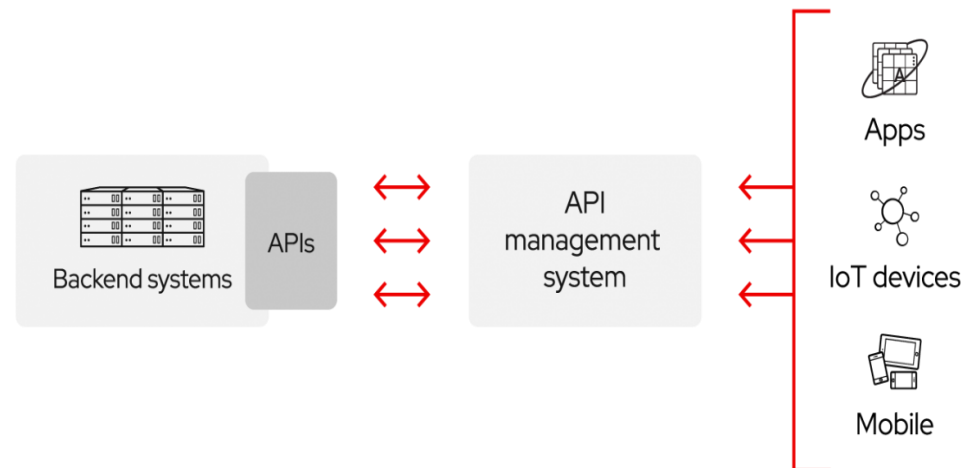
Par exemple, l'API conçue pour un service de météo peut demander à l'utilisateur de fournir un code postal et au producteur de renvoyer une réponse en deux parties : la première concernant la température maximale et la seconde la température minimale.

AVANTAGES DES APIS

- ✓ Elle permet de pouvoir interagir avec un système sans se soucier de sa complexité et de son fonctionnement. ...
- ✓ Une API est souvent spécialisée dans un domaine et sur un use case particulier ce qui simplifie son utilisation, sa compréhension et sa sécurisation.
- ✓ Les API constituent un moyen simplifié de connecter votre propre infrastructure au travers du développement d'applications cloud-native.

AVANTAGES DES APIS

- ✓ Elles vous permettent également de partager vos données avec vos clients et d'autres utilisateurs externes.
- ✓ Les API publiques offrent une valeur métier unique, car elles peuvent simplifier et développer vos relations avec vos partenaires, et éventuellement monétiser vos données (l'API Google Maps en est un parfait exemple)



API REST

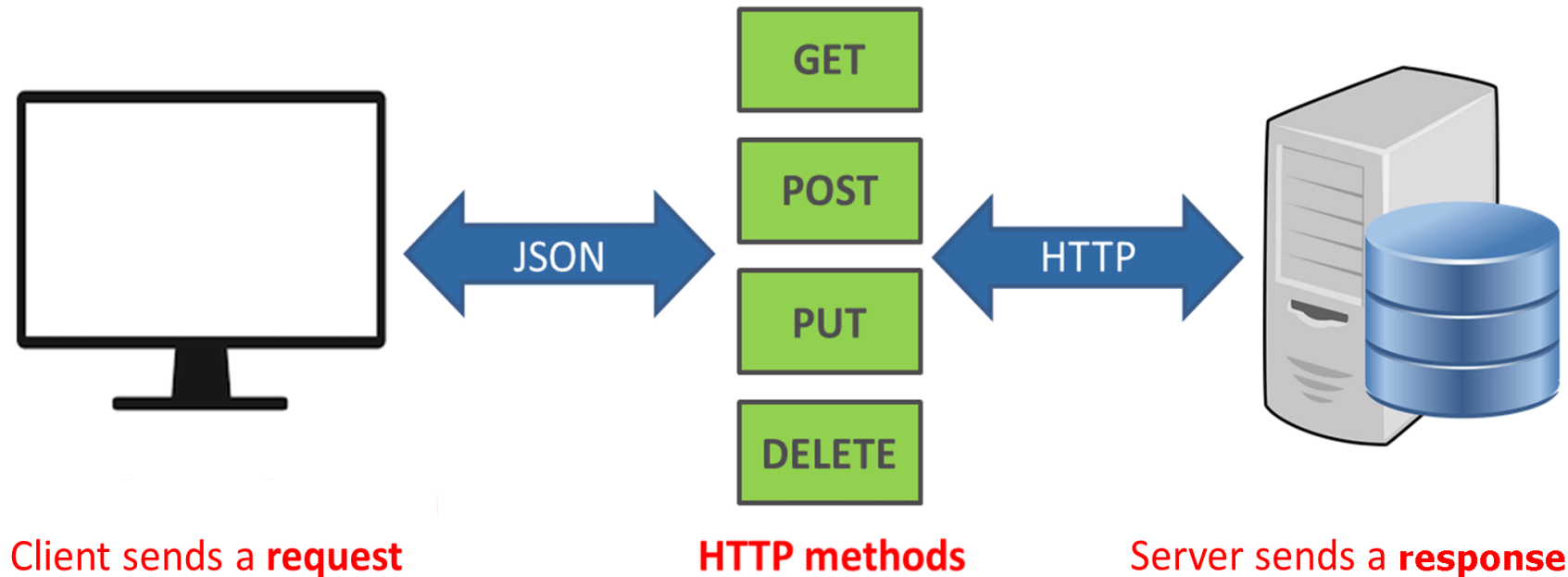
Roy Fielding a défini REST comme un style architectural et une méthodologie fréquemment utilisés dans le développement de services Internet, tels que les systèmes hypermédias distribués.

La forme complète de l'API REST est l'interface de programmation d'applications de transfert d'état représentationnelle, plus communément appelée service Web API REST.

Par exemple, lorsqu'un développeur demande à l'API Twitter de récupérer l'objet d'un utilisateur (une ressource), l'API renvoie l'état de cet utilisateur, son nom, ses abonnés et les publications partagées sur Twitter. Cela est possible grâce aux projets d'intégration d'API.

API REST

Cette représentation d'état peut être au format JSON, XML ou HTML.



COMMENT FONCTIONNE UNE API REST?

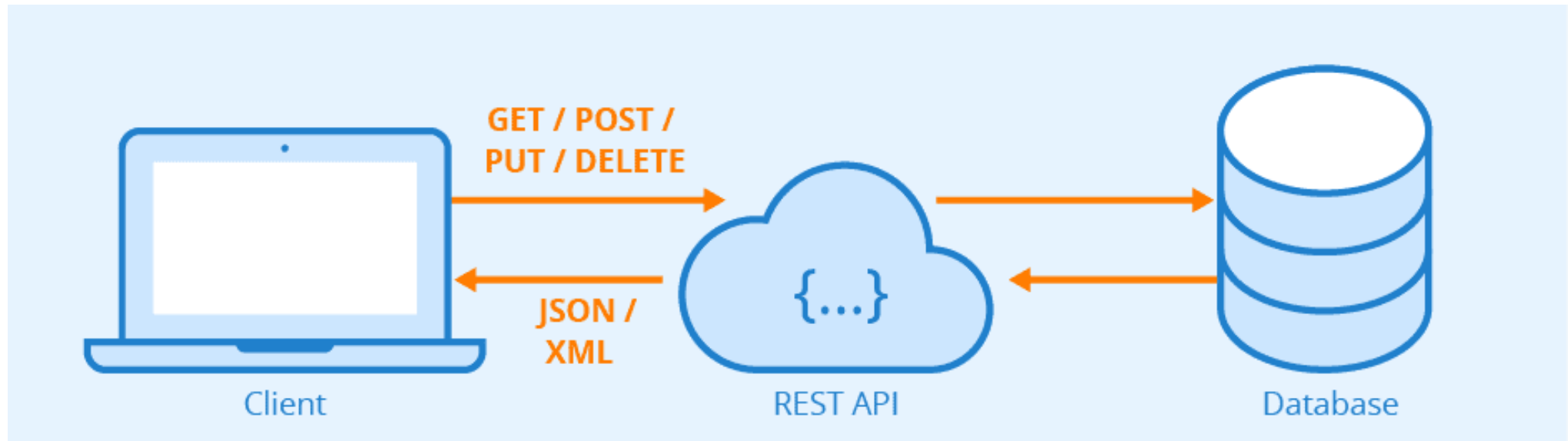
REST détermine la structure d'un API. Les développeurs s'obligent à un ensemble de règles spécifiques lors de la conception d'une API. Par exemple, une loi stipule qu'un lien vers une URL doit renvoyer certaines informations.

Chaque URL est connue sous le nom de demande (request), tandis que les données renvoyées sont appelées réponse (response).

L'API REST décompose une transaction pour générer une séquence de petits composants. Chaque composant aborde un aspect fondamental spécifique d'une transaction. Cette modularité en fait une approche de développement flexible.

COMMENT FONCTIONNE UNE API REST?

Une API REST exploite les méthodes HTTP décrites par le Protocole RFC 2616



PROTOCOLE HTTP

C'est quoi le protocole http et qui sont les méthodes utilisées par ce protocole

C'est aussi la différence entre méthode get et post

PROTOCOLE HTTP

HTTP (Hypertext Transfer Protocol) est créé pour fournir la communication entre les clients et le serveur.

Il fonctionne en tant qu'une requête et une réponse.

Il existe deux méthodes HTTP principalement utilisées: GET et POST:

La méthode GET

La méthode GET de HTTP demande des données d'une source spécifiée. Les demandes GET peuvent être mises en cache et rester dans l'historique du navigateur. Il peut également être marqué.

Il ne doit jamais être utilisé lorsque vous travaillez sur des données sensibles. Les requêtes GET ont des restrictions de longueur et ne doivent être utilisées que pour obtenir des données.

La méthode POST

La méthode POST envoie les données à traiter à une source spécifiée. Contrairement à la méthode GET, les requêtes POST ne sont jamais paramétrées, elles ne restent pas dans l'historique du navigateur et nous ne pouvons pas les mettre en signet. De plus, les requêtes POST n'ont aucune restriction de longueur de données.

PROTOCOLE HTTP

	GET	POST
Peut être marqué	Oui	Non
Peut être mise en cache	Oui	Non
Historique	Reste dans l'historique de navigateur.	Ne reste pas dans l'historique de navigateur.
Restrictions de type de données	Seulement les caractères ASCII sont autorisés.	N'a aucune restriction. Les données binaires sont également autorisées.
Sécurité	Il est moins sécurisé que le POST car les données envoyées font partie de l'URL.	Le POST est un peu plus sûr que GET car il ne reste pas dans l'historique du navigateur ni dans les journaux du serveur Web.
Visibilité	Les données sont visibles à tous dans l'URL.	N'affiche pas les données dans l'URL.

PROTOCOLE HTTP

Outre les méthodes GET et POST, il existe d'autres méthodes

Méthode	Descriptions
HEAD	Il en va de même avec la méthode GET mais elle ne renvoie que des lecteurs HTTP, pas de corps de document.
PUT	Il télécharge la représentation de l'URI spécifié.
DELETE	Il supprime la ressource spécifiée.
OPTIONS	Il retourne les méthodes HTTP supportées par le serveur.
CONNECT	Il convertit la connexion de demande en tunnel TCP / IP transparent.

NODE.JS C'EST QUOI?

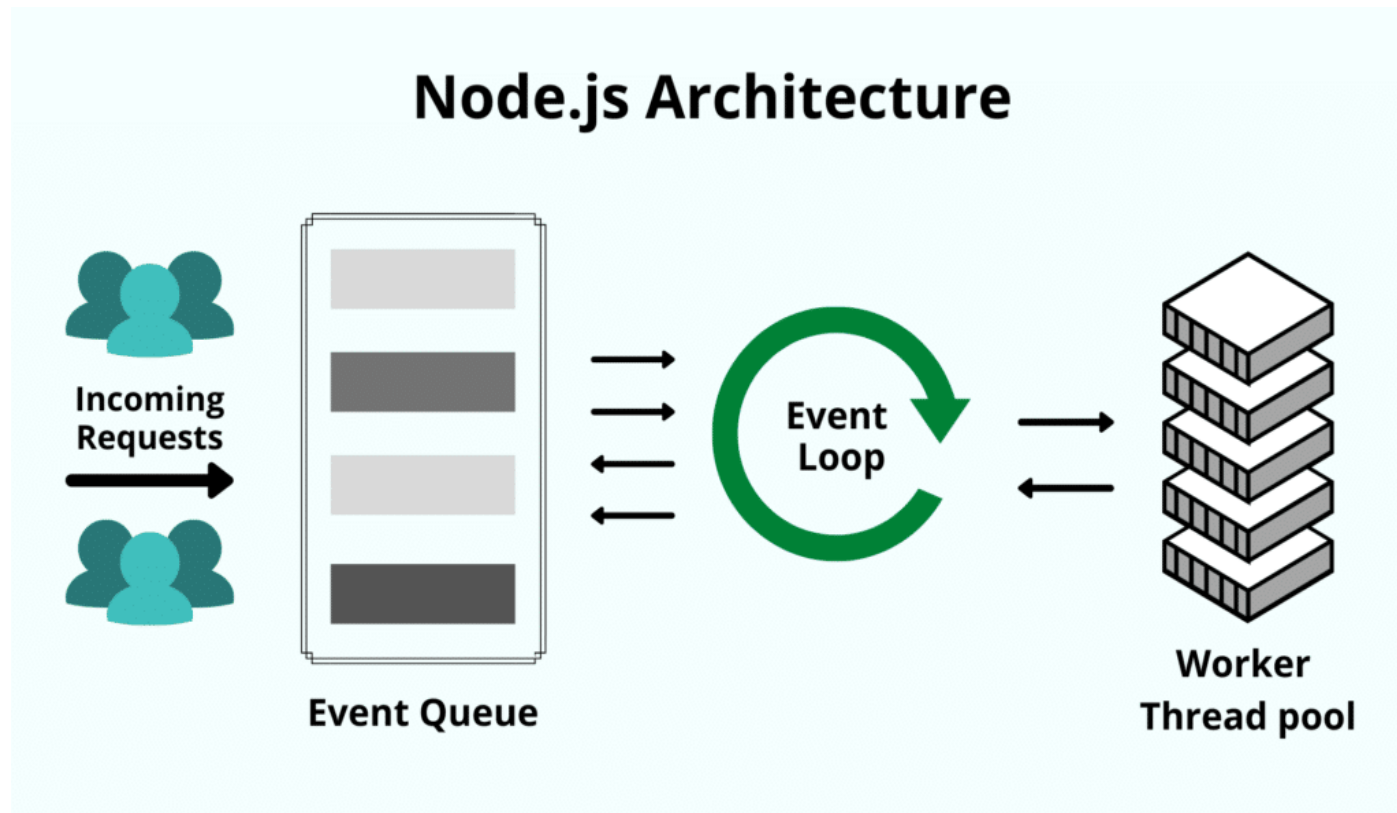
- ✓ **Node.js** est un environnement pour développer et déployer des applications web à base du Javascript.
- ✓ Plusieurs frameworks Javascript permettent de développer la partie frontale tel que Angular, VueJS , React
- ✓ **Node.js** est un environnement d'exécution single-thread, open-source et multi-plateforme permettant de créer des applications rapides et évolutives côté serveur et en réseau.
- ✓ Il fonctionne avec le moteur d'exécution JavaScript V8 et utilise une architecture d'E / S non bloquante et pilotée par les événements, ce qui le rend efficace et adapté aux applications en temps réel.



NODE.JS C'EST QUOI?

- ✓ Node.js utilise l'architecture « Single Threaded Event Loop » pour gérer plusieurs clients en même temps.
- ✓ Pour comprendre en quoi cela est différent des autres runtimes, nous devons comprendre comment les clients concurrents multi-threads sont gérés dans des langages comme Java.
- ✓ Dans un modèle requête-réponse multi-thread, plusieurs clients envoient une requête, et le serveur traite chacune d'entre elles avant de renvoyer la réponse. Cependant, plusieurs threads sont utilisés pour traiter les appels simultanés. Ces threads sont définis dans un pool de threads, et chaque fois qu'une requête arrive, un thread individuel est affecté à son traitement.

NODE.JS C'EST QUOI?



FONCTIONNALITÉS NODE.JS

Facile

- Node.js est facile à prendre en main

Evolutif

- ✓ capable de gérer un grand nombre de connexions simultanées avec un débit élevé

Vitesse

- ✓ L'exécution non bloquante des threads rend Node.js encore plus rapide et plus efficace.

Multiplateforme

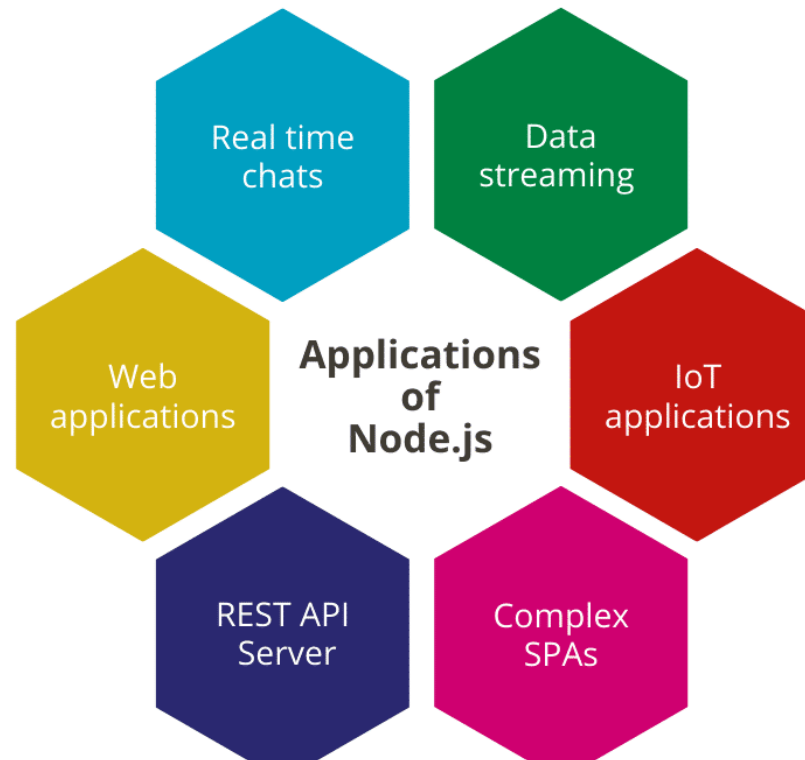
- ✓ La prise en charge multi-plateforme vous permet de créer des sites web SaaS, des applications de bureau et même des applications mobiles, le tout en utilisant Node.js.

Maintenable

- Node.js est un choix facile pour les développeurs, car le frontend et le backend peuvent être gérés avec JavaScript comme un seul langage

FONCTIONNALITÉS NODE.JS

Quelques entreprises des plus populaires qui utilisent Node.js aujourd'hui :
Twitter, Spotify , eBay, LinkedIn



EXPRESS

Express est un framework Node pour développer la partie Backend

Ainsi il est dorénavant possible de développer des applications fullstackJS

Dans ce module, on va développer des API Rest via Express, Dans un premier temps on va utiliser des données dans des fichiers JSON, ensuite on va traiter le cas MongoDB et MySQL.



CONFIGURATION DE L'ENVIRONNEMENT DE DÉVELOPPEMENT

1. Tout d'abord, télécharger le fichier Windows Installer (.msi) depuis le site officiel de Node.js.
2. Suivre le processus d'installation normal.
3. Vérifier l'installation de node JS: Ouvrir la ligne de commande et saisir la command **node -v** puis **npm -v**
4. Créer un dossier de travail testMicroservice
5. Sous le dossier **testMicroservice**, ouvrir le terminal et exécuter les commandes suivantes :

```
npm install express mongoose nodemon
```
6. Installer Postman

CONNEXION À LA BASE DE DONNÉES MONGODB

- ❑ Créer un répertoire de travail
- ❑ Lancer la commande `npm init -y`
- ❑ `Npm install express mongoose nodemon`
- ❑ Modifier le `package.json`
- ❑ Créer deux fichiers `db.js` et `index.js`
- ❑ BDD : Institut

Collections : Stagiaires

Nom, prenom, filiere annee(Number)

GET ET POST

```
app.get('/afficherparking/:id', (req, res)=> {  
    const identifi=parseInt(req.params.id)  
    const p = parkings.find(parking => parking.id ===  
    identifi)  
    res.status(200).json(p)  
  
});  
app.post('/ajouter', (req,res)=>{  
    const nv=req.body  
    parkings.push(nv)  
    res.status(200).json(parkings)  
})
```

PUT

```
app.put('/parkings/:id', (req, res) => {  
  const id = parseInt(req.params.id)  
  let parking = parkings.find(  
    parking => parking.id === id  
  )  
  parking.name = req.body.name,  
  parking.city = req.body.city,  
  parking.type = req.body.type,  
  res.status(200).json(parking)});
```



```
const mongoose=require('mongoose')
mongoose.set('strictQuery', true)

mongoose.connect("mongodb://127.0.0.1:27017/parkingDB",
    { useNewUrlParser: true,
      useUnifiedTopology: true })
    .then(() => console.log('Connexion à MongoDB réussie !'))
    .catch(() => console.log('Connexion à MongoDB échouée !'));

const ParkingSchema = mongoose.Schema(
    {
        nom:{
            type:String,
            required: true},
        type:String,
        city:String
    }
)

const Parking= mongoose.model("parkings", ParkingSchema)
module.exports = Parking
```

```
const mongoose=require('mongoose')
mongoose.set('strictQuery', true)
mongoose.connect("mongodb://127.0.0.1:27017/joueurDB",
    { useNewUrlParser: true,
      useUnifiedTopology: true
    }
  )
  .then(() => console.log('Connexion à MongoDB réussie !'))
  .catch(() => console.log('Connexion à MongoDB échouée !'));
//Creation de schéma
const JoueurSchema = mongoose.Schema(
  {
    id:Number,
    nom:String,
    poste:String
  })
//Creation de collection
const Joueur= mongoose.model("joueurs", JoueurSchema)
module.exports = Joueur
```

```
const mongoose=require('mongoose')
mongoose.set('strictQuery', true)
mongoose.connect("mongodb://127.0.0.1:27017/EquipeDB",
    { useNewUrlParser: true,
      useUnifiedTopology: true
    }
    )
    .then(() => console.log('Connexion à MongoDB réussie !'))
    .catch(() => console.log('Connexion à MongoDB échouée !'));
const EquipeSchema = mongoose.Schema(
  {
    id:Number,
    nom:String,
    country:String,

  }, { versionKey: false})
const Equipe= mongoose.model("equipe", EquipeSchema)
module.exports = Equipe
```

EXPLICATIONS

- **dbName** : le nom de la base de données à laquelle se connecter.
- **hôte** : le nom d'hôte ou l'adresse IP du serveur MongoDB.
- **port** : le numéro de port sur lequel le serveur MongoDB écoute.
- **useNewUrlParser** : définissez sur true pour utiliser le nouvel analyseur de chaîne de connexion MongoDB.
- **useUnifiedTopology** : définissez sur true pour utiliser le nouveau moteur de détection et de surveillance de serveur.
- **set(strictquery())** : garantit que les valeurs transmises à notre constructeur de modèle qui n'ont pas été spécifiées dans notre schéma ne sont pas enregistrées dans la base de données.

```
const express = require('express');
const app = express();
const Parking = require('./db')
app.use(express.json());

app.get("/parking", (req, res) => {
  Parking.find()
    .then(parkings => res.status(201).json(parkings))
    .catch(error => res.status(400).json({ error }))
})

app.listen(3000, () => {
  console.log("Serveur à l'écoute au port 3000")
});
```

LIRE LES DONNÉES À PARTIR DE LA BASE MONGODB

```
const express = require('express');
const app = express();
const Parking = require('./db')
app.use(express.json());

app.get("/parking", (req, res) => {
  Parking.find()
    .then(parkings => res.status(201).json(parkings))
    .catch(error => res.status(400).json({ error })))
})
app.listen(3000, () => {
  console.log("Serveur à l'écoute.....")
});
```

AJOUTER DES DONNÉES

```
app.post('/ajouter', (req, res) => {  
  const { id, nom, country } = req.body;  
  const nvelle_equipe = new Equipe({  
    id,  
    nom,  
    country  
  });  
  nvelle_equipe.save()  
    .then(equipe => res.status(201).json(equipe))  
    .catch(error => res.status(402).json(error));  
});
```

```
app.get("/parkings", (req, res) => {  
  Parking.find()  
    .then(parkings => res.status(201).json(parkings))  
    .catch(error => res.status(400).json({ error }));  
});  
  
app.listen(port, ()=>{  
  console.log("Serveur à l'écoute au port 3000")  
});
```



```
app.put('/parkings/:id', (req, res) => {  
  Parking.updateOne({ _id: req.params.id }, req.body )  
    .then(parkings => res.status(200).json(parkings))  
    .catch(error => res.status(400).json({ error }));  
});
```

```
app.delete('/parkings/:id', (req, res) => {  
  const { id } = req.params;  
  Parking.deleteOne({ _id: id })  
    .then(() => res.status(200).json({ message: 'Objet supprimé !' }))  
    .catch(error => res.status(400).json({ error }));  
});
```

```
app.post("/joueur/ajouter", (req, res) => {  
  const { nom, poste } = req.body;  
  const newJoueur = new Joueur({  
    nom,  
    poste  
  });  
  
  newJoueur.save()  
    .then(j => res.status(201).json(j))  
    .catch(error => { res.status(400).json(error) })  
});
```

```
app.get("/parkings/:id", (req, res) => {  
  const voitureId = req.params.id;  
  
  //findById is literally a wrapper of findOne({_id: id})  
  
  Parking.findOne({_id:voitureId})  
    .then(  
      parkings => res.status(201).json(parkings)  
    )  
    .catch(  
      error=> res.status(400).json(error)  
    )  
  });
```

TPN° 1

1. Créer une collection joueurs dans la base de données avec Express. Chaque joueur dispose des champs(id, idEquipe,nom,numero,poste)
2. Développer les opérations crud pour l'entité joueur(4 requetes)
3. Développer la route permettant d'afficher les joueurs d'une équipe via son id(de l'équipe).
4. Développer la route permettant de modifier le poste d'un joueur donné via son id.
5. Développer la route affiche les noms des joueurs qui ont un numéro dans la liste d'entrée: {numeros:[1,7,9]}
6. Développer la route qui calcule le nombre de joueurs d'une équipe donnée.

- 1-Développer les opérations crud pour l'entité joueurs (4 requetes)
- 2-Développer la route permettant d'afficher les joueurs d'une équipe via son id(de l'équipe).
- 3-Développer la route permettant d'afficher l'équipe d'un joueur donné via son id.
- 4-Développer la route permettant de chercher un joueur à partir de son nom
- 5-Développer la route permettant d'afficher les joueurs qui ont un numéro dans la liste entrée en paramètre(exemple ci-dessous)

```
const express=require('express');
const app=express();
app.use(express.json())
const equipes=require('./equipes.json')
```

```
app.get('/equipes',(req,res)=>{
    res.status(200).json(equipes)
})
```

```
app.post('/add',(req,res)=>{
    //recuperer la nouvelle donnee
    const nv_equipe=req.body
    //Ajouter la nouvelle equipe à la liste
    equipes.push(nv_equipe)
    res.status(200).json(equipes)
})
```

EXERCICE

- ❑ Considérer la base de données **ProduitDB** de MongoDB
- ❑ Créer la collection ***produits***
- ❑ Chaque *produit* dispose des champs(id, nom, prix, categorie)
- ❑ Utiliser ***mongoose*** et ***nodemon***

- 1-Développer la route qui permet d'afficher tous les produits.
- 2-Développer la route permettant de supprimer un produit via son id.
- 3-Développer la route permettant d'ajouter un nouveau produit.
- 4-Développer la route permettant de chercher un produit a partir de son nom
- 5-Développer la route permettant de modifier la catégorie d' un produit.
- 6-Développer la route permettant d'afficher les produits dont le prix est supérieur à 2000 et la catégorie différente de « **Meubles** ».
- 7-Développer la route permettant de calculer le prix total et le prix moyen des produits.

EXERCICE

- ❑ Considérer la base de données **ProduitDB** de MongoDB
- ❑ Créer la collection ***produits***
- ❑ Chaque *produit* dispose des champs(id, nom, prix, categorie)
- ❑ Utiliser ***mongoose*** et ***nodemon***

- 1-Développer la route qui permet d'afficher tous les produits.
- 2-Développer la route permettant de supprimer un produit via son id.
- 3-Développer la route permettant d'ajouter un nouveau produit.
- 4-Développer la route permettant de chercher un produit a partir de son nom
- 5-Développer la route permettant de modifier la catégorie d' un produit.